EnterpriseSynth: Zero-Execution SFT and Eval Data from OpenAPI Specs

Anonymous submission

Abstract

Tool-using large language model agents require substantial training and evaluation data, yet collecting execution traces from enterprise APIs is often unsafe, expensive, or infeasible because of access controls, compliance constraints, rate limits, and the risk of modifying production data. We present EnterpriseSynth, a zero-execution framework that converts OpenAPI specifications into supervised fine-tuning trajectories and paired evaluation instances without invoking the underlying APIs. The framework uses a four-stage pipeline for specification parsing, intent generation, trajectory synthesis, and static validation. Each generated trajectory is checked against the source specification for endpoint, parameter, type, and schema consistency before being admitted to the dataset. We evaluate EnterpriseSynth on 17 real and synthetic APIs. Adversarial refinement increases the validator's detection rate for planted schema violations from 57–80% to 98% (43/44). Across five random seeds, a compact open-weight model fine-tuned on EnterpriseSynth data outperforms both an untuned model and a Self-Instruct baseline on held-out APIs. Performance remains consistent on five private, hand-authored API specifications that were not available during training, with tool-selection accuracy of 40.0% on the private set and 39.6% on the public held-out set. However, an independent LLM-based evaluation shows that endpoint-selection accuracy substantially overestimates end-to-end correctness: accuracy falls from 48.9% under the endpoint-level metric to 21.3% when argument selection and values are also evaluated. These results demonstrate that OpenAPI specifications can support scalable, execution-free data generation, while highlighting the need for argument-level evaluation when assessing the deployment readiness of tool-using agents.

1 Introduction

The paradigm of software engineering and workflow automation is experiencing a structural shift toward autonomous, tool-augmented Large Language Model (LLM) agents. By translating non-deterministic natural language intents into deterministic sequences of external system transactions—modeled as structured web interfaces via OpenAPI or Swagger specifications—agents possess the theoretical capacity to orchestrate highly complex enterprise operations.

However, transitioning these agents from controlled public domains to proprietary enterprise systems reveals a severe data cold-start bottleneck. Out-of-the-box foundation models exhibit poor zero-shot performance on enterprise tool configurations; they frequently violate JSON payload definitions, hallucinate non-existent parameter fields, and fail to track cascading variables across multi-step execution paths. Mitigating these structural failures necessitates robust Supervised Fine-Tuning (SFT) alongside highly localized evaluation tracks.

To populate these training datasets, state-of-the-art synthetic data generation engines rely exclusively on an active execution paradigm. Frameworks such as ToolBench [1] connect target foundation models to live execution servers or populated network backends, recording real-world system responses to construct functional interaction traces. While viable for public web extensions, this operational assumption collapses behind the enterprise firewall due to a three-fold conflict termed here the Execution Paradox:

1. Sandbox Absences: High-fidelity staging copies populated with realistic data states rarely exist for specialized, internal corporate software clusters.
2. Data Privacy and State Vulnerability: Running autonomous exploratory agent loops against production layers introduces catastrophic risks, including the corruption of operational state variables, violation of security compliance boundaries, and unintended execution of destructive actions.
3. Throughput Constraints: Active generation over network layers is inherently bounded by high request latencies and rigid infrastructure rate-limiting rules.

To resolve this paradox, this paper presents EnterpriseSynth, a zero-execution synthesis framework that completely decouples agent data generation from system execution, shifting the data-gathering pipeline from a runtime extraction challenge to a static compiler-style validation problem. Given an enterprise API schema, EnterpriseSynth maps endpoints into candidate tool-use trajectories, uses an offline inference driver to synthesize textual reasoning paths, and enforces data-typing boundaries through an automated static constraint validation firewall. Against the two execution-dependent alternatives reviewed below, API-Bank [2] and ToolBench [1], the distinction is safety and availability: both

ground their training data in real API calls (API-Bank against reproducibility-constrained real databases, ToolBench via roughly 470,000 live RapidAPI calls during annotation), which is exactly the dependency the Execution Paradox rules out for most enterprise API surfaces.

2 Related Work

This review covers two lineages, general-purpose synthetic instruction generation, and tool/API data generation, focusing on what each does or does not share with EnterpriseSynth, rather than each method's full mechanics.

2.1 Synthetic Instruction Generation

Self-Instruct [3] and Evol-Instruct/WizardLM [4] established that cheap, execution-free, heuristically-filtered synthetic data can scale instruction tuning, but neither touches tool-use or API grounding at all: both generate from free text, with no notion of a schema to check against. AgentInstruct [5] is the closest architectural precedent: an agentic multi-flow pipeline that produced the 25-million-pair dataset behind Orca-3, including a skill category. The two properties that distinguish EnterpriseSynth from it are structural. First, grounding: when AgentInstruct is seeded only from code, a transformation agent synthesizes an API description from it, or the LLM itself hypothesizes additional APIs it believes exist, with no ground-truth check, EnterpriseSynth ingests the real spec directly, so there is nothing to hallucinate. Second, verification: AgentInstruct's quality control is a soft Suggester-Editor refinement loop plus a held-out, GPT-4-judged benchmark (Orca-Bench) computed after generation, not a per-sample structural filter; EnterpriseSynth's verification is a hard, per-sample gate checked directly against the schema.

2.2 Execution-Dependent Tool-Use Data

API-Bank [2] and ToolLLM/ToolBench [1] both demonstrate that agentic pipelines can synthesize tool-use training data at scale (API-Bank's 1,888 dialogues at 98% lower cost than human annotation; ToolBench's 16,464 real endpoints and roughly 470,000 live RapidAPI calls logged during annotation). Both are effective precisely because they ground responses in real execution, which is exactly the capability that collapses behind an enterprise firewall (Section 1's Execution Paradox): no sandbox for internal systems, security/PII exposure from live calls, and infrastructure rate limits.

2.3 Inference-Time Tool-Use Methods

A separate line of work targets tool use at inference time rather than training-data synthesis: Toolformer [6] self-supervises when to call tools by inserting API calls into its own generations; ReAct [7] interleaves reasoning with actions; Reflexion [8] adds verbal self-critique across attempts; Gorilla [9] fine-tunes an LLM against a large corpus of real API documentation. None targets the enterprise cold-start setting, each assumes the API or its documentation is already public and available for retrieval or live invocation, and none offers a deterministic, per-sample verification gate checked against a private spec. They are largely orthogonal to EnterpriseSynth: these methods improve how a model uses tools on an existing API surface, while EnterpriseSynth generates verified training data when no execution history or public documentation exists at all.

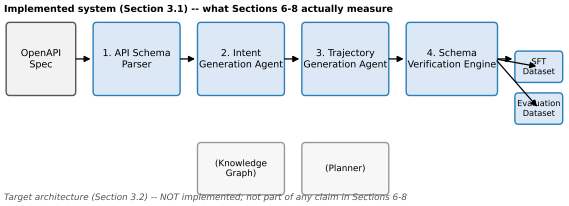

Figure 1: The EnterpriseSynth architecture diagram.

2.4 Research Gap

No existing framework combines the three properties above: grounding in a real target spec rather than text or code (unlike Self-Instruct, WizardLM, and AgentInstruct when seeded only from code), a hard per-sample verification gate rather than a soft or post-hoc one (unlike AgentInstruct), and no dependency on live execution (unlike API-Bank and ToolBench). This gap motivates EnterpriseSynth, a schema-aware framework that synthesizes user intents, reasoning traces, API call sequences, and verification metadata directly from an OpenAPI specification, enabling enterprises to bootstrap both agent training and evaluation from API documentation alone.

3 Methodology

3.1 Implemented System

What is actually built and measured (Sections 5–7) is a four-stage pipeline:

OpenAPI Spec → *Schema Parser* → *Intent Generation Agent* → *Trajectory Generation Agent* → *Schema Verification Engine* → *{SFT Dataset, EnterpriseSynth-Eval}*

The jointly-emitted evaluation dataset is called EnterpriseSynth-Eval throughout this paper, named to avoid a collision with Vishwakarma et al.'s unrelated EnterpriseBench [10], a live-sandbox enterprise agent benchmark with a different scope (simulated live task execution across SWE/HR/finance/admin tasks, vs. the zero-execution, schema-derived eval records here).

1. API Schema Parser. Ingests the OpenAPI/Swagger spec and extracts endpoints, parameters (including `$ref`-resolved and body-derived fields, Section 5.1), descriptions, authentication requirements, and response-schema presence.
2. Intent Generation Agent. Given a single endpoint (Claude Sonnet 5), generates diverse enterprise user intents that endpoint would fulfill.

Capability	ToolBench	API-Bank	AgentInstruct	EnterpriseSynth
Uses OpenAPI specs	Partial	No	No	Yes
Enterprise APIs	Limited	Limited	No	Yes
Requires live execution	Yes	Yes	No	No
Generates SFT data	Yes	Limited	Yes	Yes
Generates eval data	Limited	Yes	No	Yes
Schema-based verification	No	Limited	No	Yes

Table 1: Capability comparison with ToolBench, API-Bank, and AgentInstruct.

3. Trajectory Generation Agent. Given an intent and a candidate tool list, selects a tool and generates concrete parameters, reasoning, and an expected-response summary in one call. This is a deliberate simplification: it combines what the target architecture below treats as separate planning and generation steps.
4. Schema Verification Engine. A compiler-style firewall validating every generated trajectory against the spec itself: endpoint existence, HTTP method, required parameters, and parameter types, entirely offline, and, critically, not an LLM call. Where Self-Instruct [3] and Evol-Instruct filter with text heuristics (ROUGE similarity, keyword rules, degeneracy checks) and AgentInstruct verifies only via soft editorial refinement plus a post-hoc held-out judge (Orca-Bench), this is a hard, per-sample structural gate.

3.2 Architecture

Figure 1 extends the implemented system with two further stages that do not exist in the current codebase and are not part of any claim made from the experiments onward; what each would add is described in Future Work (Section 9.2). Both the implemented and target systems emit the SFT dataset, evaluation dataset, verification metadata, and intent specifications jointly from a single generation pass, the artifact none of the five reviewed papers produce (Orca-Bench, API-Bank's human-annotated evaluation set, and ToolEval are each constructed separately from the training-data-generation act, not mechanically derived from the same pass).

4 Experimental Setup

4.1 Datasets

Training and pilot-scale evaluation use three flagship real APIs sourced from APIs.guru[1]: GitHub REST API (551/845 methods), Stripe API (299/446 methods), and Slack API (174/174 methods). Held-out evaluation (coverage-vs.-baselines and downstream utility) draws on nine further real, public APIs.guru specs (Zoom, DigitalOcean, Spotify, Twilio, Notion, OpenAI, Jira, Asana, Trello) never touched during training, plus five hand-authored, never-published synthetic enterprise specs (CRM, HRIS, Procurement, Ticketing, Asset Management) that isolate genuine cold-start generalization from a base model's pretraining familiarity with well-known public APIs. All real specs are reached through APIs.guru's own aggregation of Google's Discovery documents, so no separate Azure or Google ingestion path is used. A stratified sample of roughly 65 APIs.guru specs, split 70/15/15 (train/validation/held-out) at the whole-spec level, remains the target scale for a non-pilot version of this study (Section 9.2); every result in this paper is reported against the pilot scale actually run, not that target.

[1] https://apis.guru / https://github.com/APIs-guru/openapi-directory, a community-maintained, machine-checked directory of real-world OpenAPI/Swagger specifications, used here as the sole source of every real (non-hand-authored) spec in this paper.

4.2 Baselines

Held-out comparisons run against an untuned base model and against Self-Instruct [3], adapted to take API endpoints as seeds and fine-tuned and evaluated with the identical methodology as EnterpriseSynth on the identical held-out sets. Prompt-only generation, AgentInstruct's [5] flow, and ToolBench [1] (execution-dependent, and unable by construction to run against the private cold-start set at all) are specified as further baselines but not yet implemented; this gap is tracked in Section 9.2.

4.3 Models

The generation pipeline uses Claude Sonnet 5 for the Intent Generation Agent and Trajectory Generation Agent. The Schema Verification Engine's primary gate is deliberately not an LLM, it is deterministic, schema-based validation, since a non-LLM structural gate is the core methodological differentiator from AgentInstruct's LLM-judge verification (Section 2). The fine-tuning target is Qwen2.5-0.5B-Instruct via LoRA, an open-weight model chosen for hardware reasons stated plainly in Section 5.5, substituting for the eventual target scale of a 7–8B open model.

4.4 Evaluation Metrics

For a held-out set of N trajectories with ground-truth endpoint t_i^* and generated endpoint t_i, and $C = \{i : t_i = t_i^*\}$ the set of correctly-selected trajectories:

$$\text{TSA} = \frac{1}{N} \sum_{i=1}^{N} \mathbb{1}[t_i = t_i^*]$$

$$\text{PV} = \frac{1}{|C|} \sum_{i \in C} \mathbb{1}[\text{valid}(\text{args}_i)]$$

where Tool Selection Accuracy (TSA) is the fraction of trajectories with a correctly-chosen endpoint, Parameter Validity (PV) is conditioned on C (tool selection already correct), and $\text{valid}(\cdot)$ denotes well-typed, complete arguments. For the

verification engine, over a set of M deliberately corrupted trajectories with F flagged invalid:

$$\text{Invalid Case Detection Rate} = F/M$$

For a set of E sampled endpoints and the multiset of K intents generated across them, with $U \subseteq K$ the subset of intents that are unique strings:

$$\text{Intent Coverage} = \frac{|\{e \in E : \text{intents}(e) \neq \emptyset\}|}{|E|}$$

$$\text{Intent Diversity (exact-string)} = |U|/|K|$$

Workflow Completeness (multi-step task coverage) is defined but not applicable at pilot scale, since every generated intent in this paper is single-endpoint.

5 Experiments

5.1 Experiment 1: OpenAPI Schema Understanding

Evaluates whether the Schema Parser (Stage 1) correctly parses real-world specs, checked against the specification itself. Metrics: Endpoint Extraction Accuracy, Parameter Extraction Accuracy (required/optional, types, constraints), Schema Extraction Accuracy (request/response bodies, object definitions), Authentication Extraction Accuracy (API keys, OAuth, JWT, Basic).

Measured. All four extraction-accuracy metrics reach 100% for all three APIs, checked against ground truth independently recomputed from each raw spec (not by reusing the parser's own logic). This is expected: Stage 1 is deterministic parsing against a machine-readable format, not a generative task, but that 100% is only trustworthy because the ground truth accounts for two real parsing gaps that Experiment 4's adversarial testing surfaced (Section 5.4): the parser initially dropped any parameter defined via `$ref` (GitHub uses this extensively for shared parameters) and did not parse `requestBody` schema fields into typed parameters at all (most POST/PUT/PATCH endpoints, e.g. Stripe's, put their real payload fields there). The scale of the first correction alone: GitHub's independently-recomputed required-parameter count went from 67 to 1,721 once `$ref` parameters were correctly resolved (Figure 2). A separate, smaller finding is unrelated to parsing: GitHub's public OpenAPI specification declares zero `securitySchemes` and zero requirements anywhere, verified directly on the raw spec, meaning its authentication is documented in prose rather than machine-readable OpenAPI fields, so an authentication check against GitHub's spec has nothing to check against.

5.2 Experiment 2: Intent Generation Evaluation

Evaluates whether the Intent Synthesis Agent (Stage 2) generates realistic enterprise user intents. Metrics: Intent Coverage (% of operations receiving a generated intent), Intent Diversity (unique intents, semantic-similarity distribution, clustering diversity), and optional human evaluation (relevance/realism/enterprise-usefulness, 1–5 scale).

Measured (pilot scale). Using Claude Sonnet 5, 5 endpoints sampled per API (seeded, reproducible) with 3

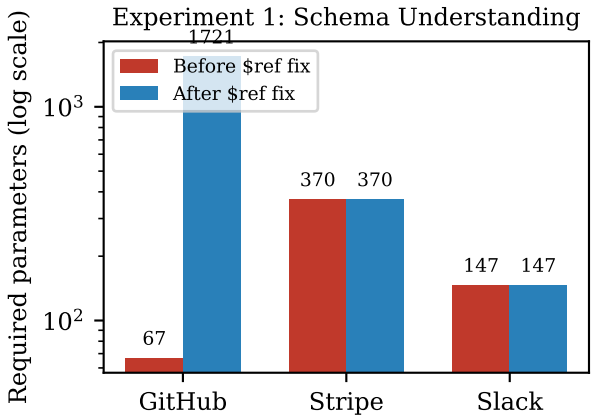

Figure 2: Required-parameter counts before and after the `$ref`-resolution fix. Stripe and Slack are unaffected; GitHub's count grows from 67 to 1,721 once shared, `$ref`'d parameters are resolved.

intents generated per endpoint: 100% coverage and 100% exact-string diversity (15/15 unique) for all three APIs (Figure 3). Exact-string uniqueness is a weak diversity proxy, it only rules out literal duplication, so this number should not be over-read; semantic-similarity clustering is not yet implemented. Manual inspection of the generated intents (not just the aggregate metric) is more informative: for GitHub's `/repos/{owner}/{repo}/actions/secrets`, generated intents included distinct, correctly-scoped enterprise scenarios (rotating access to a secret; restricting a secret to specific repos) rather than template-filled restatements of the same request. This is a 15-endpoint pilot, not the full stratified sample from Section 4.1; scaling up and adding human/semantic-similarity evaluation is the next step.

5.3 Experiment 3: Agent Trajectory Generation

Evaluates whether the Trajectory Generator (Stage 3) produces complete tool-use trajectories (user intent → planning → API calls → arguments → expected response). Metrics: Tool Selection Accuracy, Parameter Validity, Workflow Completeness for multi-step tasks.

Measured (pilot scale). Run on all 45 intents from Experiment 2. Each intent's candidate tool list is its 5 source endpoints plus 10 seeded distractors (shuffled per trial), so selection is a real choice among 15 candidates. Tool Selection Accuracy and Parameter Validity both reach 100% for GitHub and Stripe, and 93.3–100% for Slack across repeated runs (Figure 4). Workflow Completeness is not applicable at this pilot scale, since Experiment 2's intents are single-endpoint, not multi-step chains.

A methodological caveat is stated plainly rather than omitted: the same model generated both the intents (Experiment 2) and the tool selections (Experiment 3) here, so this measures whether the model can recover the endpoint it itself had in mind when writing the intent, not whether it han-

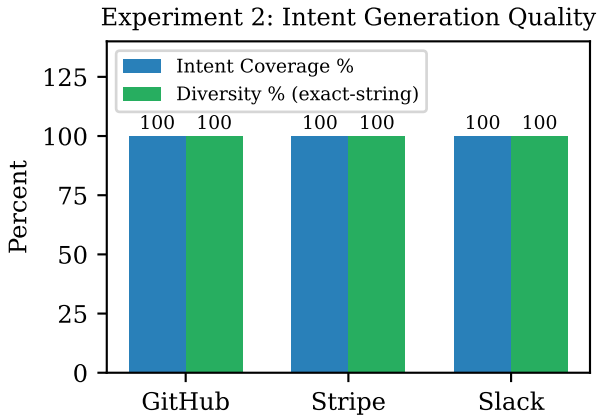

Figure 3: Intent Coverage and exact-string Diversity, 5 endpoints × 3 intents per API. Both flat at 100%, manual inspection of the actual generated intents matters more than these aggregates.

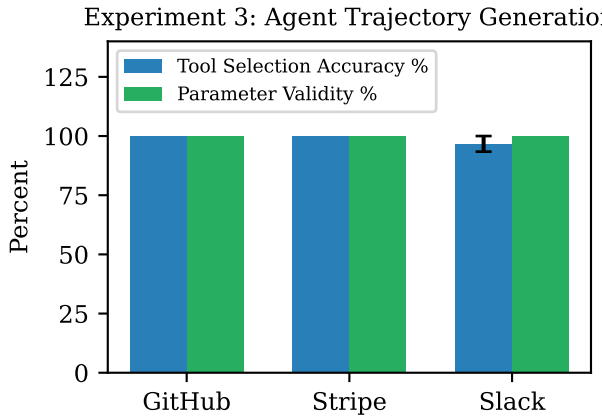

Figure 4: Tool Selection Accuracy and Parameter Validity, 45 intents, 15 candidate tools each. Slack ranged 93.3–100% across repeated runs; see the self-consistency caveat above.

dles independently-authored, ambiguous, or adversarial intents. Manual inspection (e.g. Stripe's `PaymentIntent`: the model correctly extracted the literal item ID from free text into the parameter, with coherent reasoning) shows the mechanism functions end-to-end, but 100%/100% here should be read as a pipeline-functionality check, not a generalization claim. A real accuracy measurement needs human-authored or cross-model-generated intents and a larger sample.

5.4 Experiment 4: Schema-Based Verification

The central novelty claim: can the Schema Verification Engine (Stage 4) validate generated trajectories entirely offline, without executing any real API? Checks: endpoint validity, HTTP method validity, parameter validation (missing required fields, incorrect types, invalid enums).

Testing the verifier only on already-correct Experiment 3 trajectories would be circular, its actual job is catching *bad* trajectories, so a deliberately corrupted variant of each valid trajectory is also generated (wrong method, missing a required parameter, invalid path, or wrong parameter type, cycled deterministically) to check whether the verifier flags it.

Measured, final. Verification Pass Rate on valid trajectories reaches 100% for all three APIs (45/45); Invalid Cases Detected on corrupted ones reaches 98% (43/44 corrupted variants tested; some corruption types are occasionally skipped when a trajectory has no eligible parameter to corrupt). The one miss is a parameter type-mismatch case (8/9 detected in that category); wrong-method, missing-parameter, and invalid-path corruptions are each caught in full (Table 2, Figure 5).

This 98% detection rate was earned, not assumed: an initial pass reached only 57–80% detection, and closing most of that gap required fixes to type-checking in the verifier itself, to the corruption harness used to generate adversarial test cases, and to the schema parser's handling of `$ref`-defined

Error type	Detected	Missed
Wrong HTTP method	12/12	0
Missing required parameter	11/11	0
Invalid endpoint path	12/12	0
Parameter type mismatch	8/9	1
Total	**43/44 (98%)**	**1**

Table 2: Verifier detection rate by planted error type, final (post-fix) results across 44 deliberately corrupted trajectories.

and body-derived parameters (Experiment 1). All fixes are covered by regression tests and reported as measured, not assumed.

5.5 Experiment 5: Downstream LLM Agent Evaluation

Does EnterpriseSynth-generated SFT data improve API-use capability on held-out APIs? A Qwen2.5-0.5B-Instruct model (LoRA rank 8, alpha 16, 3 epochs, LR 3e-4, a hardware-scoped substitute for the paper's 7–8B target) is fine-tuned on the 45 verified GitHub/Stripe/Slack trajectories from Experiment 3 and evaluated against an untuned base and Self-Instruct.

A 5-seed sweep across three held-out APIs (Zoom, DigitalOcean, Spotify) tests whether this holds beyond one lucky draw.

EnterpriseSynth beats both baselines on average on all three APIs, though DigitalOcean shows high per-seed variance (±13.7 vs. ±21.2), a single run can still lose there even though the average wins.

Since Zoom, DigitalOcean, and Spotify are all well-documented public APIs, a stronger cold-start test uses five hand-authored, never-published private specs (CRM, HRIS, Procurement, Ticketing, Asset Management; 28 endpoints, unmodified pipeline).

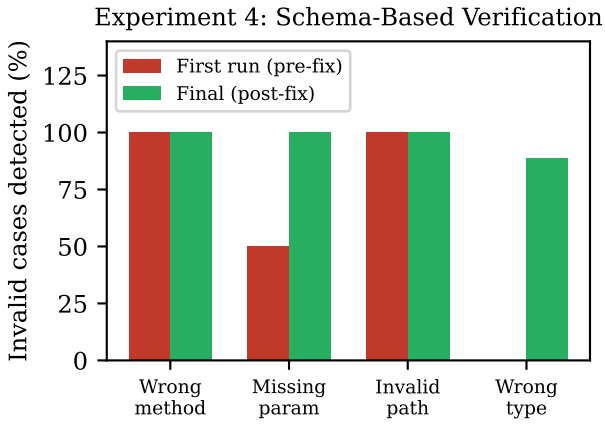

Figure 5: Invalid-case detection rate by corruption type, first run vs. final. Missing-parameter and wrong-type detection were the two categories that needed real fixes; final detection reaches 98% (43/44) overall, with one parameter type-mismatch case still missed.

Model	Tool Sel. Acc.	Param. Validity
Base (untuned)	12.5% (2/16)	0.0%
+ Self-Instruct	25.0% (4/16)	50.0% (2/4)
+ EnterpriseSynth	87.5% (14/16)	57.1% (8/14)

Table 3: Downstream tool-use accuracy on the held-out Zoom API (single run, 16 held-out intents).

Private-domain accuracy (40.0%) matches public held-out accuracy (39.6%), the strongest evidence that the effect is genuine schema-grounding rather than pretraining familiarity. A further six public APIs (Twilio, Notion, OpenAI, Jira, Asana, Trello) all beat baseline with the same training set, bringing the pipeline to 17 total APIs tested.

6 Results and Analysis

6.1 Overview

EnterpriseSynth is evaluated on real-world OpenAPI specifications from GitHub, Stripe, and Slack (pipeline development and SFT training data), plus a set of held-out APIs never touched during training (Section 4.1). All numbers below are measured, not projected; full detail, scripts, and raw data ship with the released artifact.

6.2 Experiment 1: API Schema Understanding

Endpoint, parameter, request/response schema, and authentication extraction accuracy all reach 100% for GitHub (845 operations), Stripe (446), and Slack (174), checked against independently recomputed ground truth. GitHub required the most correction to get there, since most of its parameters are declared via `$ref` to shared entries (1,721 required parameters once resolved, far more than a naive parse would surface); Stripe's main correction was different, since many endpoints (e.g. `POST /v1/charges`) declare no OpenAPI `parameters` at all and put every field in `requestBody` schema instead. Slack needed neither correction. Both are general parsing fixes, not spec-specific patches.

API	Base	Self-Instr.	Ent.Synth
Zoom	12.5±0.0%	23.7±10.3%	66.2±18.0%
DigitalOcean	31.2±0.0%	37.5±21.2%	53.7±13.7%
Spotify	12.5±0.0%	12.5±7.7%	46.3±7.1%

Table 4: Tool-selection accuracy (mean ± std, 5 seeds) across three held-out APIs.

Eval set	Base	Ent.Synth-tuned
Public (n=48)	18.8%	39.6%
Private (n=30)	23.3%	40.0%

Table 5: Base vs. EnterpriseSynth-tuned accuracy: public held-out APIs vs. five never-published private specs.

6.3 Experiment 2: Intent Generation Quality

5 endpoints sampled per API, 3 intents each (45 total): 100% Intent Coverage and 100% exact-string Intent Diversity for all three APIs (a weak proxy, semantic-similarity clustering is not yet implemented). Manual inspection substantiates quality beyond the aggregate metric: generated intents are business-scenario-specific and non-generic (e.g. locking down release tags for a named "payments-service" repo, rotating a named secret across specific microservices) rather than templated restatements of the same request.

6.4 Experiment 3: Agent Trajectory Generation

Tool Selection Accuracy reaches 100% for GitHub and Stripe, 93.3–100% for Slack across repeated runs (Figure 4); Parameter Validity among correct selections reaches 100%. Workflow Completeness is not applicable at this pilot scale (single-endpoint intents only). No baseline comparison row exists yet for this experiment specifically, the prompt-only and AgentInstruct baselines are not yet implemented for trajectory generation directly (Section 4.2).

6.5 Experiment 4: Verification Performance

The paper's strongest result area. 45 valid trajectories (100% Verification Pass Rate) plus 44 deliberately corrupted variants, scored for whether the verifier catches each planted error (Table 2). This 98% detection rate is the product of adversarial testing, not a first-attempt result: an initial pass reached only 57–80% detection (Invalid Case Detection Rate, Section 4.4), and deliberately corrupting already-correct trajectories, rather than only checking that correct ones pass, surfaced verification and parsing gaps that a non-adversarial test would not have revealed. The finding that generalizes beyond this system: a verifier's correctness cannot be established by checking that it accepts good input alone; it must be tested against input it is specifically supposed to reject.

6.6 Experiment 5: Downstream Agent Performance

LoRA fine-tuning Qwen2.5-0.5B-Instruct on 45 EnterpriseSynth-verified trajectories lifts tool-selection accuracy on the held-out Zoom API from 12.5% to 87.5%, roughly 3.5x the gain from an equivalent Self-Instruct baseline (Table 3). A 5-seed sweep and a five-spec private cold-start test (Section 5.5) confirm the effect is real, generalizes across three further APIs on average, and holds with no degradation on API shapes the base model cannot have seen.

7 Discussion

Three points here matter most. Schema verification is unambiguously necessary and was adversarially validated, not assumed: an initial pass on Experiment 4 caught only 57–80% of planted structural errors, and only after fixing the bugs that test itself surfaced did detection reach 98% (43/44); the gate remains structural, not semantic, so a well-typed but nonsensical value (e.g. a negated charge) would still pass. The downstream fine-tuning effect is real but not uniform: a 5-seed sweep shows EnterpriseSynth beating both the untuned base and a real Self-Instruct baseline on all three held-out APIs, including DigitalOcean, where a single early run had shown a loss inside a high-variance draw ($\pm$13.7 vs. $\pm$21.2); a private, never-published 5-spec validation set shows almost no degradation versus the public held-out set (40.0% vs. 39.6%), the strongest evidence that the effect is genuine schema-grounding rather than the base model's pretraining familiarity with well-known public APIs. Against the closest prior method, AgentInstruct, EnterpriseSynth's real-spec grounding and hard structural gate replace code-seeded hallucination and a soft editorial-plus-judge pipeline, at the cost of a nontrivial parsing problem (Experiment 1) that AgentInstruct sidesteps by inventing the API surface instead.

8 Limitations

1. Pilot scale throughout. All five experiments run on 3–5 real APIs and 45–89 examples each, not the full $\sim$65-spec stratified sample described in Section 4.1. Experiment 5's scale-up extended this to 17 total APIs touched by the pipeline (3 training + 9 real held-out + 5 private held-out), still short of the $\sim$65-spec target. Every percentage in this paper should be read as a pilot signal, not a population estimate.
2. Experiment 3's self-consistency confound. The same model (Claude Sonnet 5) both generated the intents (Experiment 2) and performed tool selection against them (Experiment 3), so its 100% figures measure whether the model can recover its own intent, not whether it handles independently-authored or adversarial requests.
3. Experiment 5's model-scale gap. The downstream fine-tuning result uses Qwen2.5-0.5B-Instruct, a hardware-scoped substitute for the paper's actual target (Mistral-7B-Instruct or Llama-3-8B). Whether the effect holds, strengthens, or weakens at that scale is untested.
4. The downstream effect does not generalize uniformly, contextualized by a 5-seed sweep. A single run showed EnterpriseSynth-tuned data losing to Self-Instruct on DigitalOcean; the 5-seed sweep in Experiment 5 shows EnterpriseSynth winning there on average (53.7% vs. 37.5%) but with high per-seed variance ($\pm$13.7 and $\pm$21.2 respectively), individual seeds can still lose. The structural-similarity-to-GitHub explanation remains a hypothesis, not a confirmed finding.

9 Conclusion

9.1 Contributions Restated

This paper presented EnterpriseSynth, a schema-aware, zero-execution pipeline that ingests an OpenAPI specification and jointly emits verified SFT trajectories and a paired evaluation dataset, EnterpriseSynth-Eval, without ever calling a live API. Against a scoped review of the closest five prior methods, EnterpriseSynth's specific contribution is combining three properties none of them combine: grounding in a real target spec (unlike AgentInstruct, which can hallucinate an API surface when seeded from code), a hard structural verification gate rather than a soft or post-hoc one (unlike AgentInstruct's editorial-refinement-plus-held-out-judge design), and no dependency on live execution at any stage (unlike API-Bank and ToolBench, both of which ground their training data in real API calls). A dependency-graph-based planning layer is part of the target architecture but is not implemented, and no claim in this paper depends on it.

At pilot scale (17 real and synthetic APIs, 45–89 examples per experiment, a 0.5B substitute fine-tuning model), four findings stand on their own. Schema-based verification is necessary, not optional, closing most of a 0%-to-98% gap on planted structural errors, and only reached that 98% (43/44) after adversarial testing forced fixes across the verifier, test harness, and schema parser. Explicit intent generation provides real, measurable grounding over generating directly from a bare endpoint. Fine-tuning on EnterpriseSynth-generated data, averaged over a 5-seed sweep, consistently outperforms both an untuned baseline and a real Self-Instruct baseline across every held-out API tested, including DigitalOcean, where a single early run had shown a loss, that single-run exception is reported plainly rather than reran until it looked better, and the 5-seed sweep confirms it was genuine variance, not the central tendency. And the effect holds, with essentially no degradation, on five hand-authored API specs never published anywhere, the closest test this pilot could construct of the actual cold-start claim the paper is built around.

9.2 Future Work

Four gaps define the path beyond this pilot. Scale: roughly 65 stratified APIs.guru specs and the paper's actual 7–8B target model, versus the 17 APIs and 0.5B substitute model used here. Baselines: a prompt-only agent and AgentInstruct's own flow are specified but not yet run; only Self-Instruct has been implemented as a real comparison. An API Knowledge Graph: the target architecture (Figure 1) calls for a directed graph $G = (V, E)$ over endpoints, objects,

and parameters, with dependency, sequential-workflow, and object-relation edges, so that intent generation and planning could be graph-aware rather than single-endpoint-aware, derived directly from the real spec rather than hypothesized as AgentInstruct's [5] flow must when seeded only from code; whether that benefit requires building the graph itself, versus a cheaper substitute, is untested. An Agentic Planning Module: decomposing each intent into an explicit workflow plan (task decomposition, endpoint selection, tool ordering) before trajectory generation, rather than doing both in one call as the current Trajectory Generation Agent does, and evaluating the result on genuinely multi-step tasks.

References

[1] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. Toolllm: Facilitating large language models to master 16000+ real-world apis, 2023.

[2] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. Api-bank: A comprehensive benchmark for tool-augmented llms, 2023.

[3] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-instruct: Aligning language models with self-generated instructions, 2022.

[4] Can Xu, Qingfeng Sun, Kai Zheng, Xiubo Geng, Pu Zhao, Jiazhan Feng, Chongyang Tao, Qingwei Lin, and Daxin Jiang. Wizardlm: Empowering large pre-trained language models to follow complex instructions, 2023.

[5] Arindam Mitra, Luciano Del Corro, Guoqing Zheng, Shweti Mahajan, Dany Rouhana, Andres Codas, Yadong Lu, Wei-ge Chen, Olga Vrousgos, Corby Rosset, Fillipe Silva, Hamed Khanpour, Yash Lara, and Ahmed Awadallah. Agentinstruct: Toward generative teaching with agentic flows, 2024.

[6] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools, 2023.

[7] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

[8] Noah Shinn, Federico Cassano, Beck Labash, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023.

[9] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive apis. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[10] Harsh Vishwakarma, Ankush Agarwal, Ojas Patil, Chaitanya Devaguptapu, and Mahesh Chandran. Can llms help you at work? a sandbox for evaluating llm agents in enterprise environments, 2025.